

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Gál Gergő Károly

2026

Szegedi Tudományegyetem
Informatikai Intézet

Tollaslabda Versenykezelő Rendszer
fejlesztése

Development of a Badminton Tournament
Management System

Szakdolgozat

Készítette:

Gál Gergő Károly

Gazdaságinformatikus BSc szakos
hallgató

Témavezető:

Dr. Bodnár Péter

egyetemi adjunktus

Szeged

2026

Feladatkiírás

A szakdolgozat célja egy olyan modern webalkalmazás megtervezése és megvalósítása, amely átfogó informatikai megoldást nyújt a kisebb és közepes méretű amatőr tollaslabda-versenyek teljes körű szervezéséhez és operatív lebonyolításához. A MERN-stack technológiai alapjaira épülő rendszernek biztonságos, JSON Web Token (JWT) alapú hitelesítéssel kell védenie a versenyszervezők adatait, biztosítva a többfelhasználós környezetben az adatok szigorú elkülönítését és a tulajdonosi jogosultságok ellenőrzését. A fejlesztés során kiemelt figyelmet kell fordítani a versenyek, kategóriák és játékosok rugalmas kezelésére, kiegészítve a nevezési díjak adminisztrációjával és az egyesületi alapú csoportos fizetések támogatásával. A hallgató feladata olyan folyamatok kialakítása, amelyek támogatják a helyszíni jelenlét-regisztrációt, garantálva, hogy a sorsolás és a mérkőzés-generálás kizárólag a ténylegesen megjelent indulók alapján történjen meg. A rendszernek automatizálnia kell a csoportkörös (round-robin) és az egyenes kieséses (playoff) lebonyolítási formákat, alkalmazva a klasszikus polygon-rotációs vagy cirkuláns gráfalapú algoritmusokat a párosítások és a pihenőnapok egyenletes elosztásához. Az alkalmazásnak a Nemzetközi Tollaslabda Szövetség (BWF) hivatalos szabályait követve kell végeznie a szettekre bontott eredményrögzítést és validációt, beleértve a kötelező kétpontos különbség és a maximális pontkorlát ellenőrzését is. Az üzleti logikának automatikusan kell kalkulálnia a csoportállásokat és feloldania a holtversenyeket az egymás elleni eredmények, szett- és pontkülönbségek alapján, miközben egy mohó algoritmussal támogatja a pályák és mérkőzések ütemezését a játékosok kötelező pihenőidejének figyelembevételével. A szoftvernek transzparens módon, publikus kijelzőnézeten keresztül kell tájékoztatnia a résztvevőket, továbbá biztosítania kell az adatok hordozhatóságát CSV formátumú exportálási lehetőséggel. A projekt eredményeként egy modularitást és tesztelhetőséget szem előtt tartó, működőképessé szoftverrendszert kell átadni, amely megfelel a specifikált funkcionális és nem-funkcionális követelményeknek.

Tartalmi összefoglaló

- **A téma megnevezése:**

Tollaslabda versenykezelő rendszer fejlesztése

- **A megadott feladat megfogalmazása:**

A szakdolgozat célja egy olyan webalapú alkalmazás megtervezése és megvalósítása, amely támogatja a tollaslabda versenyek szervezését és lebonyolítását. A rendszer feladata a versenyek, kategóriák, játékosok és nevezések kezelése, a jelenlét-regisztrációs folyamat támogatása, a csoportkörös és egyenes kieséses lebonyolítás kezelése, az eredmények rögzítése, az állások kiszámítása, valamint a mérkőzések ütemezése.

- **A megoldási mód:**

A feladat megoldása egy háromrétegű webalkalmazás formájában történt. A rendszer React-alapú kliensoldali felületből, Node.js és Express alapú szerveroldali alkalmazásból, valamint MongoDB adatbázisból áll. A frontend és a backend közötti kommunikáció REST API-n keresztül, JSON formátumú adatokkal valósul meg.

- **Alkalmazott eszközök, módszerek:**

A fejlesztés során React, Vite, Node.js, Express, MongoDB, Mongoose, JWT-alapú hitelesítés, REST API, CSV export, valamint automatizált smoke tesztek kerültek alkalmazásra. A rendszer fő üzleti logikái külön szolgáltatási rétegben valósultak meg, ideértve a round-robin párosításgenerálást, az eredményvalidációt, az állásszámítást, a többszintű holtverseny-kezelési irányelveket (policies), a rájátszásgenerálást és az ütemezést.

- **Elért eredmények:**

A fejlesztés eredményeként létrejött egy működőképes, demonstrálható webalkalmazás, amely képes versenyek és kategóriák kezelésére, játékosok és nevezések rögzítésére, jelenlét-regisztráció kezelésére, csoportkörös mérkőzések generálására, eredmények szettekre bontott rögzítésére, állásszámításra, rájátszás generálására, mérkőzések ütemezésére, auditnaplózásra és CSV formátumú adatexportokra.

- **Kulcsszavak:**

webalkalmazás, tollaslabda, versenykezelés, MERN-stack, REST API, ütemezés.

Tartalomjegyzék

Feladatkiírás.....	3
Tartalmi összefoglaló.....	4
Tartalomjegyzék	5
BEVEZETÉS	7
1. A PROBLÉMA ÉS SZAKMAI HÁTTÉR.....	9
1.1. A tollaslabda versenyek lebonyolításának sajátosságai	9
1.2. Csoportkörös és egyenes kieséses rendszerek.....	10
1.3. A versenyszervezés gyakorlati problémái.....	11
2. TECHNOLÓGIAI HÁTTÉR ÉS VÁLASZTOTT ESZKÖZÖK	12
2.1. Webalkalmazások általános felépítése	12
2.2. A MERN-alapú megközelítés.....	13
2.3. A választott technológiák szerepe a rendszerben	13
3. KÖVETELMÉNYSPECIFIKÁCIÓ	16
3.1. Funkcionális követelmények	16
3.2. Nem-funkcionális követelmények.....	18
3.3. Elfogadási kritériumok.....	19
3.4. A jelen verzió határai	20
4. TERVEZÉS.....	21
4.1. Architektúra	21
4.2. Adatmodell	22
4.3. API terv	23
4.4. Kulcslogikák és algoritmusok	24
4.4.1. Csoportkörös mérkőzések generálása	24

4.4.2. Sorsoláslezárás és jelenlét alapján kialakított mezőny	25
4.4.3. Eredményvalidáció	25
4.4.4. Csoportállás és holtversenyek kezelése	26
4.4.5. Továbbjutók és egyenes kieséses szakasz generálása	26
4.4.6. Ütemezési logika	26
4.4.7. Állapotkezelés és ütközésvédelem	27
5. MEGVALÓSÍTÁS	29
5.1. Backend modulok felelősségei	29
5.2. Frontend fő nézetek és állapotkezelés	30
5.3. Integráció: adatáramlás, hibakezelés és validáció	32
5.4. A megvalósítás összegzése	33
6. TESZTELÉS ÉS VALIDÁCIÓ	34
6.1. Tesztstratégia	34
6.2. Tesztesetek a követelményekhez kötve	34
6.3. Kritikus helyzetek és "Edge case"-ek kezelése	36
6.4. A tesztelés eredményeinek összessége	37
7. ÖSSZEGZÉS ÉS KITEKINTÉS	38
7.1. Az elért eredmények összefoglalása	38
7.2. A rendszer erősségei	38
7.3. Korlátok és hiányosságok (Tervezési kompromisszumok)	39
7.4. Továbbfejlesztési lehetőségek	39
7.5. Személyes és szakmai tanulságok	40
7.6. Záró összegzés	41
Irodalomjegyzék	42
Nyilatkozat	43
Mesterséges intelligencia használatáról szóló nyilatkozat	44
Köszönetnyilvánítás	45

BEVEZETÉS

A szakdolgozatom témája egy webalapú tollaslabda-versenykezelő rendszer tervezése és megvalósítása. A témaválasztásom nem volt véletlen. A tollaslabda gyermekkorom óta meghatározó része az életemnek. Akár versenyzőként álltam a pályán, akár edzőként vagy táborszervezőként segédkeztem, pontosan láttam, mennyi háttérmunkával és szervezéssel jár egy-egy esemény lebonyolítása.

A szoftverfejlesztés ötlete egy konkrét eset, egy házi verseny szervezése során fogalmazódott meg bennem. Akkoriban egy olyan általános szoftvert használtunk, amely nem kifejezetten erre a sportágra készült. Bár az alapvető feladatokat meg tudtuk vele oldani, folyamatosan korlátokba ütköztünk az olyan tipikus problémáknál, mint egy játékos késése, meg nem jelenése, egy sérülés miatti visszalépés, vagy éppen egy elszámolt és utólag javítandó szetteredmény. Ezek a gyakorlati tapasztalatok mutattak rá, hogy egy kisebb, amatőr verseny is rengeteg olyan adminisztratív lépést és gyors döntést igényel, amit egy célzottan erre fejlesztett alkalmazás jelentős mértékben megkönnyíthet.

Célom tehát egy olyan működőképes, webalapú alkalmazás elkészítése volt, amely egyetlen, átlátható felületen fogja össze a versenyszervezés fázisait az előkészületektől egészen az eredményhirdetésig. Kifejezetten a kisebb, amatőr és házi bajnokságok igényeit tartottam szem előtt. Ezeken az eseményeken a szervezőknek gyorsan, sokszor időnyomás alatt kell reagálniuk a változásokra, így nincs szükségük egy robusztus, de túlbonyolított platformra.

A megvalósított rendszer végigkíséri a felhasználót a teljes folyamaton. Kezeli a versenylétrehozást és konfigurálását, kategóriákat és a hozzájuk tartozó egyedi beállításokat, a nevezéseket és a helyszíni jelenlétet, majd a tényleges indulók alapján legenerálja a csoportkörös, illetve az egyenes kieséses mérkőzéseket. A sportági szabályoknak megfelelő eredményrögzítésen túl a szoftver elvégzi az automatikus állásszámítást, és támogatja a pályák, valamint a mérkőzések ütemezését is, kiegészítve olyan hasznos funkciókkal, mint a játékvezetői rotáció vagy az auditnaplózás.

Technológiai oldalról az alkalmazás a modern kliens-szerver architektúrát követve, a MERN-stack (MongoDB, Express, React, Node.js) alapjaira épül. A böngészőből elérhető kliensfelület és a háttérrendszer REST API-n keresztül, JSON formátumban kommunikál egymással, a biztonságos hozzáférést pedig JWT-alapú hitelesítés garantálja. Annak érdekében, hogy a rendszer jól karbantartható és bővíthető legyen, a kritikus üzleti logikákat, mint amilyen a sorsolás, az eredményvalidáció vagy a holtversenyek feloldása, különálló szerveroldali modulokba szerveztem.

A dolgozat felépítése a fejlesztés logikai ívét követi. Az első fejezet a versenyszervezés gyakorlati kihívásait és szakmai hátterét mutatja be, majd a választott technológiák és a követelményrendszer ismertetése következik. A negyedik és ötödik fejezet a rendszer tervezésének (architektúra, adatmodell, API) és megvalósításának részleteibe nyújt betekintést. Végül a tesztelés és validáció tapasztalatait, az elért eredményeket, a szoftver korlátait és a jövőbeli továbbfejlesztési lehetőségeket összegzem.

1. A PROBLÉMA ÉS SZAKMAI HÁTTÉR

A tollaslabda versenyek szervezése első ránézésre egyszerű feladatnak tűnhet. Össze kell gyűjteni a résztvevőket, mérkőzéseket kell lejátszani, és ki kell hirdetni az eredményeket. A gyakorlatban azonban egy közepes méretű verseny is néhány kategóriával, párhuzamos pályákkal és több tucat játékosal meglehetősen összetett adminisztrációs feladatot jelent. Ez a fejezet bemutatja, hogy a tollaslabda versenyek lebonyolítása milyen sajátos kihívásokat tartogat, és ezek hogyan indokolják egy szoftveres megoldás fejlesztését.

1.1. A tollaslabda versenyek lebonyolításának sajátosságai

Laikus szemmel egy tollaslabda-mérkőzés csupán két játékos vagy páros összecsapása, maga a versenyszervezés azonban jóval összetettebb feladat. Ahogy nő a létszám és a kategóriák száma, az adminisztráció hirtelen kritikus tényezővé válik.

Egy eseményen belül jellemzően több, egymástól teljesen független kategória indul (például férfi egyes, vegyes páros vagy amatőr mezőny). Ezek informatikai szempontból önálló egységek, saját sorsolással, mérkőzésekkel és eredménylistával rendelkeznek. Egyetlen ömlesztett játékoslista vezetése a gyakorlatban hamar átláthatatlanná tenné a szervezést.

További sajátosság az eredmények rögzítése. A tollaslabdában nem csupán egy végső pontszámot adminisztrálunk, hanem szetteket, amelyek érvényességét a Nemzetközi Tollaslabda Szövetség (BWF) szigorú szabályai kötik [14]. Egy hivatalos mérkőzés jellemzően két nyert szettig tart, ahol egy szett megnyeréséhez 21 pontot kell elérni úgy, hogy az ellenféllel szemben legalább kétpontos különbség alakuljon ki, pontegyenlőség esetén pedig a szett maximális pontkorlátja 30 pont [14]. A szoftvernek tudnia kell értelmezni ezeket a feltételeket, hiszen egy elért eredmény később a teljes csoportállást és a továbbjutást is felboríthatja.

Ehhez társul az idő- és pályakezelés kihívása. A mérkőzések párhuzamosan zajlanak a korlátozott számú pályákon, és a szervezőnek folyamatosan figyelnie kell a játékosok pihenőidejére, valamint a pályaütközések elkerülésére. Végül pedig a papírforma ritkán egyezik a valósággal. A nevezési lista szinte sosem azonos a tényleges indulókkal, folyamatosan előfordulnak késések, meg nem jelenések vagy visszalépések. Egy valóban hasznos versenykezelőnek nemcsak a tökéletes forgatókönyvet, hanem ezeket az operatív, helyszíni anomáliákat is kezelnie kell.

1.2. Csoportkörös és egyenes kieséses rendszerek

Az amatőr, kis és közepes méretű versenyeken leggyakrabban a csoportkörös, az egyenes kieséses, illetve ezek kombinált formáját alkalmazzák.

A csoportkör (körmérkőzéses rendszer) legnagyobb előnye, hogy a résztvevők számára több mérkőzést garantál, így egyetlen vereség nem jelenti a verseny végét. Hátránya viszont az extrém időigényessége. Nagyobb mezőny esetén emiatt gyakran részleges körmérkőzést alkalmaznak, ahol nem mindenki játszik mindenkivel. Bár ez időt spórol, komoly párosítási logikát követel meg a szoftvertől a terhelés kiegyenlítése érdekében.

A csoportkör legkényesebb pontja az állásszámítás és a holtversenyek feloldása. Ha két vagy több játékos azonos mutatókkal zár, az egymás elleni eredmény, a szett- és pontkülönbség, vagy többes körbeverés esetén a szűkített tabella dönt. Kézi számolásnál ez a vitás helyzetek egyik legfőbb forrása.

Az egyenes kieséses rendszer ezzel szemben tiszta és gyors. A győztes továbbjut, a vesztes búcsúzik. Hogy a legjobb játékosok ne az első körökben ejtsék ki egymást, a párosításokat általában kiemelés alapján készítik el.

A legnépszerűbb és a bemutatott szoftver által is támogatott megoldás a kettő kombinációja. A játékosok egy csoportkörben kezdik meg a versenyt, majd a legjobbak az egyenes kieséses ágon folytatják a küzdelmet.

1.3. A versenyszervezés gyakorlati problémái

A kisebb versenyek lebonyolítása ma is gyakran manuális, papíralapú formában vagy általános táblázatkezelő programokban (például Excel) történik. Amíg a mezőny kicsi, ez működhet. Amint párhuzamos kategóriák indulnak, vagy egy kategórián belül már nagyobb létszámú versenyző van, a manuális módszerek nehezen alkalmazhatók.

A legfőbb probléma az adatszinkronizáció hiánya. Ha egy visszalépést külön kell átvezetni a sorsoláson, a mérkőzéslistán és a pénzügyi elszámoláson, szinte borítékolható a hiba. Bár az Excel rugalmas, szoftveres értelemben teljesen passzív. Nem akadályozza meg, hogy egy játékost véletlenül egyszerre két pályára osszanak be, és nem dob hibaüzenetet egy szabálytalan (például 21-20-as) szetteredményre sem.

Az ütemezés szintén állandó stresszforrás. A tervezett menetrend percek alatt felborulhat egy-egy elhúzódó, szoros háromszettes mérkőzés miatt. Ha a nyilvántartás nem képes a tényleges csúszások alapján újrakalkulálni a várható kezdési időket, az információs káoszhoz vezet. A játékosok és edzők folyamatos tájékoztatást igényelnek, ami feleslegesen terheli a szervezőket. Ezt a helyzetet egy nyilvános, automatikusan frissülő kijelzőnézet azonnal orvosolja.

Végül nem szabad megfeledkezni a pénzügyi adminisztrációról sem. Egy-egy egyesület gyakran egyben fizeti be a játékosai nevezési díjait. Ezeknek a csoportos tranzakcióknak az összekapcsolása a konkrét indulókkal manuálisan nehézkes, ráadásul szétszórt papírokon az utólagos visszakereshetőség (például egy vitás esetről) gyakorlatilag lehetetlen.

Ezek a mindennapos, életszerű problémák adták a motivációt a fejlesztésre, hogy egy olyan célszoftvert hozzak létre, amely leveszi a számítási és adminisztrációs terhet a szervezők válláról.

2. TECHNOLÓGIAI HÁTTÉR ÉS VÁLASZTOTT ESZKÖZÖK

A rendszer fejlesztésének megkezdése előtt az egyik legfontosabb feladatomban a megfelelő technológiai együttes kiválasztása volt. A döntést alapvetően a projekt célja és a megvalósítandó funkciók komplexitása határozta meg. Egyértelmű volt, hogy egy telepítést nem igénylő, böngészőből futtatható webalkalmazásra van szükség, amely képes kiszolgálni a versenyszervezéshez kapcsolódó, folyamatosan változó adatokat és a bonyolult üzleti logikákat.

2.1. Webalkalmazások általános felépítése

A modern webalkalmazások döntő többsége, így az általam fejlesztett versenykezelő rendszer is, kliens-szerver architektúrára épül. Ebben a felépítésben a szerver (backend) felelős az adatok tárolásáért, a jogosultságok ellenőrzéséért és az üzleti logika (például a sorsolás vagy az állásszámítás) futtatásáért, míg a kliens (frontend) a felhasználói felületet jeleníti meg.

A két réteg kommunikációját a ma már iparági szabványnak tekinthető REST (Representational State Transfer) architektúra stílusú API-n keresztül valósítottam meg. Ebben a rendszerben az egyes erőforrások (mint egy kategória vagy egy mérkőzés) egyedi URL-címeken érhetők el, az adatok cseréje pedig JSON formátumban történik.

A felhasználói élmény maximalizálása érdekében a kliensoldalt egyoldalas alkalmazásként (SPA - Single Page Application) terveztem meg. Ennek lényege, hogy a böngésző nem tölti újra a teljes weboldalt minden egyes kattintásnál, csupán a szükséges adatokat kéri le a szervertől, és dinamikusan frissíti a felületet. Egy adminisztratív felületen, ahol a szervező folyamatosan navigál a játékosok listája, az ütemezés és a csoportállások között, ez a gördülékeny működés elengedhetetlen.

2.2. A MERN-alapú megközelítés

Több alternatíva (például LAMP stack, vagy a backend esetében Python/Django, illetve Java/Spring) mérlegelése után a MERN-stack mellett döntöttem. A kliensoldal megtervezésekor kezdetben felmerült az Angular keretrendszer használata is, azonban annak robusztus, nagyvállalati környezetre szabott architektúráját túlzónak ítéltam ehhez a projekthez. Mivel a célom egy gyors, letisztult, és a versenyszervezők számára felesleges vizuális zajoktól mentes, intuitív adminisztratív felület megalkotása volt, a React mellett döntöttem. A React deklaratív és könnyedebb megközelítése nem kényszerített rám merev kereteket, így a saját logikám szerint alakíthattam ki a felületet.

A MERN mozaikszó a MongoDB, Express, React és Node.js technológiákat foglalja magába. A választásom elsődleges szakmai indoka az volt, hogy a tollaslabda-verseny adatstruktúrái - szettekre bontott mérkőzések, beágyazott konfigurációk, változó méretű csoportok - természetes módon illeszkednek a MongoDB dokumentumorientált modelljéhez, elkerülve a relációs adatbázisoknál tapasztalható objektum-relációs impedancia-eltérést. Emellett ez a környezet lehetővé teszi a teljes alkalmazás egyetlen nyelven, a rendszerem esetében modern, ECMAScript szabványú JavaScript nyelven történő megírását, ami felgyorsította a fejlesztést, és megkönnyítette az adatstruktúrák, valamint a validációs logikák összehangolását a kliens és a szerver között. A backend alapját az aszinkron, esemény vezérelt Node.js [2] és a minimalista Express keretrendszer [3] adja, amely ideális a JSON formátumú [8] adatokat mozgó REST API-végpontok kiszolgálására. Az adattárolást a dokumentumorientált MongoDB [4] és a Mongoose ODM [5] párosával oldottam meg, amely rugalmas, dokumentum-alapú struktúrák kezelését teszi lehetővé. A kliensoldali felületet a komponensalapú React [1] segítségével építettem fel, a projekt modern és gyors kiszolgálását pedig a Vite [6] eszköz biztosította.

2.3. A választott technológiák szerepe a rendszerben

A fent bemutatott technológiák nem öncélúan, hanem a tollaslabda-versenykezelés sajátos kihívásaira adott válaszként kerültek a rendszerbe.

Az adatbázis-technológia kiválasztásakor a döntő érv nem kizárólag a NoSQL sémamentessége volt, hanem a MERN-architektúra (MongoDB, Express, React, Node.js) által biztosított egységes adatszerkezet kihasználása [8]. Mivel a teljes rendszer

kliens- és szerveroldalon egyaránt JavaScript alapú, a JSON formátumú dokumentumok natív adatbázis-leképezése kiküszöböli a relációs adatbázisoknál tapasztalható objektum-relációs impedancia-eltérést. Így az olyan hierarchikus és változó méretű adatszerkezetek, mint a szettekre bontott mérkőzések és a hozzájuk tartozó validációs szabályok, egyetlen összefüggő dokumentumként kezelhetők, elkerülve a felesleges és teljesítményt rontó táblaillesztéseket. Ugyanakkor a Mongoose használatával biztosítottam, hogy a rugalmasság ne menjen az adatbiztonság rovására. Szigorú sémadefiníciókat, típusellenőrzéseket és beépített validációs szabályokat alkalmaztam az adatbázisrétegben [5].

A szerveroldalon az Express moduláris felépítése tette lehetővé, hogy a komplex üzleti logikákat, mint a párosításgenerálás, az eredményvalidáció, a csoportállás kiszámítása vagy a mérkőzések ütemezése ne a végpontokat kezelő fájlokba sűrítsem bele, hanem különálló, jól tesztelhető szolgáltatási modulokba szervezzem ki [3].

A kliensoldalon a React komponensalapú felépítése azért bizonyult kulcsfontosságúnak, mert a felületen rengeteg állapotfüggő nézet jelenik meg [1]. Teljesen más adatokat és űrlapokat kell mutatni egy még csak konfigurációs fázisban lévő kategóriánál, mint egy már zajló, vagy egy teljesen lezárt versenynél. A React ezeket a dinamikus, állapotfüggő nézetfrissítéseket hatékonyan kezeli [1].

Végül fontos megemlíteni a biztonsági megfontolásokat is. Mivel a rendszerben a versenyadatok szigorúan a létrehozó felhasználóhoz kötöttek, egy robusztus hitelesítési mechanizmusra volt szükség. Ezt a JSON Web Token (JWT) szabvány implementálásával oldottam meg [7]. A bejelentkezést követően a kliens egy tokent kap, amelyet a böngésző helyi tárolójában (localStorage) őriz, és minden védett hívásnál elküld a szervernek. Mérnöki szempontból elengedhetetlen kitérni arra, hogy ez a megközelítés az OWASP biztonsági ajánlásai [16] alapján sebezhetőséget jelenthet az úgynevezett kódbeszúrásos (Cross-Site Scripting) támadásokkal szemben. Bár a React keretrendszer beépített adatkezelési logikája magas szintű védelmet nyújt a klasszikus DOM-injektálásos támadások ellen, a harmadik féltől származó csomagok sebezhetőségei továbbra is kockázatot jelenthetnek a helyi tárolóra nézve. Ennek iparági, robusztusabb alternatívája a "httpOnly" attribútummal ellátott süti (cookie) használata lenne. Ugyanakkor a rendszer komplexitásának alacsonyan tartása érdekében, a referencia-alkalmazás keretein belül a localStorage alapú megközelítés is elfogadható kompromisszumnak bizonyult. Ez az állapotmentes hitelesítési forma

kiválóan illeszkedik a választott REST architektúrához [9], megkönnyítve az egyértelmű jogosultságkezelést.

3. KÖVETELMÉNYSPECIFIKÁCIÓ

A fejlesztendő rendszer célja egy olyan webalapú tollaslabda-versenykezelő alkalmazás létrehozása volt, amely képes végigkísérni a szervezőket a lebonyolítás teljes folyamatán. A követelmények meghatározásakor tudatosan kerültem az általános, minden sportágra ráhúzható funkciók specifikálását. Ehelyett kifejezetten a tollaslabda versenyek sajátosságaira és a gyakorlati szervezés során fellépő problémákra fókuszáltam, figyelembe véve a csoportkörös és egyenes kieséses rendszereket, a szettekre bontott eredményrögzítést, valamint a holtversenyek kezelését.

3.1. Funkcionális követelmények

A rendszer alapvető funkcionális elvárása egy biztonságos, hitelesített munkamenet-kezelés kialakítása volt, hiszen az alkalmazást egyszerre több versenyszervező is használhatja a saját eseményeihez. Ennek megfelelően a felhasználóknak lehetőségük van a regisztrációra, a bejelentkezésre és profiljuk kezelésére, amelyet a JSON Web Token (JWT) szabvány [7] garantál. Ennél is kritikusabb azonban a szigorú adatelkülönítés. A rendszer minden adatbázis-műveletnél ellenőrzi a tulajdonosi viszonyt, teljesen kizárva, hogy bárki illetéktelenül férhessen hozzá mások adataihoz. Erre a biztonságos alapra épül a versenyek létrehozása, amelyek legfelső szintű entitásként globális erőforrásokat és szabályokat, például a rendelkezésre álló pályák számát, a becsült mérkőzésidőket és a játékvezetői névsort tartalmazzák. Mivel egy eseményen belül eltérő versenyszámok futnak, a szoftver kategóriaszintű paraméterezést is biztosít, így egyaránt képes kiszolgálni a tisztán csoportkörös, a csak egyenes kieséses, illetve a kettőt kombináló lebonyolítási formátumokat.

A strukturális beállításokat követően a versenyszervezés elengedhetetlen része az átfogó nevezési és pénzügyi adminisztráció. A rendszer nemcsak a játékosok sporttechnikai adatait tárolja, hanem a nevezési díjakat és fizetési módokat is, kiegészítve a fizetési csoportok kezelésével. Ez utóbbi funkció teszi lehetővé, hogy az egyesületi, tömeges tranzakciókat a szervező egyetlen kattintással adminisztrálhassa, a szoftver pedig automatikusan frissítse az összes érintett nevezés státuszát. A valós versenyhelyzetekhez igazodva a lebonyolítás operatív fázisa a helyszíni jelenlét-regisztrációval indul. Szigorú elvárás, hogy a sorsolás és a mérkőzésgenerálás kizárólag a ténylegesen megjelent játékosok alapján történjen. A rendszer ennek alapján

automatikusan generálja le a párosításokat, ügyelve a duplikációk elkerülésére, az önmaga ellen játszó versenyzők kizárására, valamint csonka körmérkőzés esetén a mezőnyön belüli egyenletes terheléelosztásra.

A mérkőzések elindulásával a fókusz a precíz eredményrögzítésre és az ütemezésre helyeződik át. A szoftver szettekre bontva kéri be a pontokat, majd szerveroldalon validálja azokat a Nemzetközi Tollaslabda Szövetség (BWF) hivatalos szabályai [14] szerint, ellenőrizve a kötelező kétpontos különbséget és a maximális pontkorlátot. Az ütemező modul ehhez szorosan kapcsolódva rendeli hozzá a mérkőzéseket a szabad pályákhoz és a becsült időpontokhoz, figyelembe véve a kötelező pihenőidőket, beépített védelemmel megakadályozva a pálya- vagy játékosütközéseket a mérkőzések elindításakor. A befejezett mérkőzések alapján a rendszer azonnal, az aktuális adatok alapján frissíti a csoportállásokat. Kiemelt funkcionális követelmény a holtversenyek algoritmikus, szabályalapú feloldása, amely az egymás elleni eredményt, a szett- és pontkülönbséget, illetve többes körbeverés esetén az érintett játékosok szűkített tabelláját veszi alapul.

A csoportkörök lezárulta után a rendszer a kategória beállításainak megfelelően automatikusan kiválasztja a továbbjutókat, és felépíti az egyenes kieséses ágot (playoff). Fontos tervezési döntés a lezárt versenyek kontrollált utólagos korrekciója. Bár az adatok alapértelmezetten zároltak, egy adminisztratív feloldó funkció révén az esetleges emberi hibák utólag is javíthatók maradnak. A szoftver ezen kritikus lépéseinek nyomon követhetőségét egy belső auditnapló biztosítja. Végül a helyszíni kommunikációt egy hitelesítés nélkül is elérhető nyilvános kijelzőnézet támogatja. A külső adatfeldolgozást és archiválást a mérkőzések, játékoslisták és csoportállások CSV formátumú exportálhatósága teszi teljessé.

3.2. Nem-funkcionális követelmények

A konkrét funkciókon túl a rendszer minőségét, megbízhatóságát és ergonómiáját szigorú nem-funkcionális elvárások határozták meg a szoftvermérnöki alapelvekkel összhangban [12]. Az egyik legfontosabb szempont a hibatűrő működés és a robusztusság biztosítása volt. Ennek értelmében a frontendnek és a backendnek egyaránt stabilan kell kezelnie az érvénytelen kéréseket. Egy hibás URL-paraméter (például egy nem létező versenyzonosító) vagy egy rossz sorrendben végrehajtott felhasználói művelet nem okozhat szerveroldali összeomlást. Az API-nak a REST architektúra elveinek megfelelően minden esetben kontrollált HTTP-hibakódokkal és értelmezhető válaszokkal kell reagálnia [9].

A technikai stabilitás mellett kiemelt figyelmet kapott az alacsony kognitív terhelés és a magas szintű használhatóság [12]. Mivel a versenyszervezők gyakran stresszes, zajos környezetben használják a szoftvert, az elvárások között szerepelt egy egyértelmű és letisztult felület megalkotása. Ennek érdekében a kritikus, visszafordíthatatlan műveletek jól láthatóan elkülönülnek a felületen, a hibaüzenetek pedig egységesen, magyar nyelven tájékoztatják a felhasználót a probléma okáról és a megoldási lehetőségekről.

A megbízható működés további alapfeltétele az adatkonzisztencia és az integritás fenntartása. Ez a gyakorlatban azt jelenti, hogy egy mérkőzéseredmény utólagos felülírásának azonnal, láncreakciószerűen frissítenie kell a kapcsolódó csoportállást és a továbbjutási feltételeket. Végül a rendszer teljesítményével és reakcióképességével szemben támasztott elvárás, hogy a szoftver tipikus amatőr versenyméreteket mellett is azonnali válaszidővel hajtsa végre az alapvető lekérdezéseket. Emellett a számításigényesebb feladatoknak, mint amilyen a globális ütemezés vagy a csonka körmérkőzéses sorsolás generálása, szintén észrevehető várakozási idő nélkül kell lefutniuk [12].

3.3. Elfogadási kritériumok

A rendszer akkor tekinthető a specifikált követelmények szempontjából elfogadhatónak, ha az alkalmazás teljes életciklusa logikai és technikai hibák nélkül végrehajtható. Az elvárt munkafolyamat első lépéseként a felhasználónak képesnek kell lennie a regisztrációra és a bejelentkezésre, miközben a szoftver a hitelesítési protokollok révén garantálja, hogy a szervező kizárólag a saját adataihoz férjen hozzá [7]. A biztonságos belépést követően alapvető elvárás, hogy a felületen gördülékenyen létrehozható legyen új verseny és kategória, a résztvevő játékosok pedig egyenként vagy akár tömegesen is rögzíthetők legyenek a rendszerben.

A tényleges versenynap kezdetekor a sorsoláslezárás során a rendszernek a helyszíni jelenlét alapján, kizárólag a tényleges indulók figyelembevételével kell legenerálnia a csoportkörös mérkőzéseket. A lebonyolítás során az eredményrögzítésnek szigorúan a sportágspecifikus validációs szabályoknak megfelelően kell működnie a Nemzetközi Tollaslabda Szövetség (BWF) előírásaira támaszkodva [14], majd a helyesen rögzített pontok alapján a csoportállásnak automatikusan, külső beavatkozás nélkül kell frissülnie. A csoportkör lezárultát és a továbbjutók kiválasztását követően a szoftvernek önállóan fel kell építenie a megfelelő kiemelésű egyenes kieséses ágot.

Az operatív lebonyolítás szempontjából kritikus elfogadási feltétel, hogy az ütemező modul képes legyen a mérkőzéseket szabad pályákhoz és időpontokhoz rendelni. Ennek során a rendszernek automatikusan figyelembe kell vennie az érintettek kötelező pihenőidejét, és szoftveres szinten kell megakadályoznia az esetleges pályaütközéseket. Végül az adminisztrációs folyamat lezárásaként a verseny legfontosabb adatainak, így a mérkőzéseknek, a játékosok listájának és a végső állásoknak külső feldolgozás céljából, hordozható formátumban is exportálhatónak kell lenniük.

3.4. A jelen verzió határai

Egy szoftver specifikációjának elengedhetetlen része annak rögzítése, hogy mi nem tartozik a projekt hatókörébe. Mivel az alkalmazás szakdolgozati keretek között, egy konkrét amatőr célközönség számára készült, a piacon fellelhető professzionális, országos szövetségi szintű rendszerek bizonyos funkciói tudatosan kimaradtak a fejlesztésből.

A lebonyolítási formátumokat tekintve a rendszer a csoportkörös és az egyenes kieséses ágakra fókuszál. Szakmai döntés eredményeként a szoftver nem támogatja a svájci rendszert vagy a vigaszágas sorsolást, mivel ezek implementálása az amatőr események esetében szükségtelenül túlbonyolította volna a szoftveres párosítási logikát és a felületet. A jogosultságkezelés terén a jelenlegi verzió nem tartalmaz hierarchikus, összetett szerepköralapú (Role-Based Access Control) felépítést. Nincsenek különálló játékvezetői vagy pénztárosi fiókok, az adatok hozzáférése kizárólag a versenyt létrehozó tulajdonoshoz kötött.

A professzionális szoftverekkel ellentétben a pénzügyi és adminisztratív modul tekintetében az alkalmazás nem rendelkezik beépített, automatizált fizetési átjárókkal, így a bankkártyás nevezésidő-kezelés nem megoldott. A rendszer ezen felül nem végez automatikus adatszinkronizációt külső nemzeti sportági adatbázisokkal vagy hivatalos ranglistarendszerekkel. Technológiai és infrastrukturális szempontból az alkalmazás nem támogatja az elsődlegesen offline, internetkapcsolat nélküli működést.

Továbbá nem rendelkezik hardveres integrációval, vagyis nem kapcsolódik közvetlenül a pályák mellett elhelyezett digitális eredményjelzőkhöz és játékvezetői okos eszközökhöz. A dokumentumkezelés terén nem készít hivatalos, nyomtatható PDF dokumentumokat, az adatok kinyerése jelenleg CSV formátumra korlátozódik.

Ezek a határok nem szoftveres hiányosságok, hanem egy fókuszált, stabil és a célközönség számára azonnal használható referencia-alkalmazás elkészítése érdekében meghozott mérnöki tervezési döntések eredményei.

4. TERVEZÉS

A követelményspecifikációban rögzített elvárások alapján a következő lépés a rendszer architektúrális, adatbázis-szintű és algoritmikus megtervezése volt. Tervezési alapelveként fektettem le, hogy az alkalmazás funkcionálisan jól elkülönülő rétegekből épüljön fel, így az összetett versenylogika (mint például a sorsolás vagy a holtversenyek számítása) ne keveredjen közvetlenül sem a felhasználói felülettel, sem az adatbázis-hozzáféréssel.

4.1. Architektúra

A megoldás egy klasszikus, háromrétegű webalkalmazás-architektúrát követ, amely a prezentációs (frontend), az alkalmazáslogikai (backend) és a perzisztencia (adatbázis) rétegekre tagolódik.

A prezentációs réteget egy React-alapú egyoldalas alkalmazás (SPA) alkotja. Ennek tervezésekor a legfőbb szempont az volt, hogy a versenyszervezés nem egy egyszeri, lineáris folyamat, hanem egy összetett munkafolyamat, ahol a felhasználó folyamatosan vált a különböző nézetek között. Tervezési sajátosság, hogy a kliensoldalon nem egy kész, külső router-könyvtárat alkalmaztam, hanem egy saját, könnyített útvonalkezelőt írtam. Ennek az volt a határozott célja, hogy a dinamikus útvonalakból már kliensoldalon kiszűrjem a hibás azonosítókat. Így a rendszer már a böngészőben megakadályozza, hogy a felhasználó érvénytelen kontextusba navigáljon, csökkentve ezzel a felesleges és hibára futó szerveroldali hívásokat. A megoldás egyben lehetőséget teremtett a böngésző History API-jának mélyebb megismerésére külső függőség bevezetése nélkül, ami a projekt tanulási céljait is szolgálta. A frontend és a backend között egy dedikált API-szolgáltatási réteg teremt kapcsolatot, amely egységesíti a HTTP-hívásokat, automatikusan hozzáfűzi a JWT hitelesítési fejléct, és a szerveroldali hibaüzeneteket a végfelhasználó számára is érthető, magyar nyelvű visszajelzéseké alakítja.

Az alkalmazáslogikai réteg egy Node.js és Express alapú REST API. A backend tervezésekor szigorúan elválasztottam egymástól az útvonalkezelőket és az üzleti logikát. Míg a végpontok csupán a HTTP-kérések fogadásáért, a bemeneti adatok alapvető validálásáért és a jogosultságok ellenőrzéséért felelnek, addig a komplex üzleti feladatokat (sorsolás, állásszámítás, ütemezés) dedikált szolgáltatási modulokba

szerveztem ki. Ez a fajta elkülönítés tette lehetővé a kód tisztán tartását és a logikák önálló tesztelhetőségét.

A perzisztencia réteget a MongoDB adja, amelyet a Mongoose ODM segítségével illesztettem a rendszerbe. Bár a NoSQL alapvetően sémamentes, a Mongoose segítségével szigorú típusellenőrzéseket, enum-alapú korlátozásokat és indexeléseket kényszerítettem ki az adatok konzisztenciája érdekében.

4.2. Adatmodell

Az adatmodell kialakításakor a tollaslabda-versenyek sajátos fogalmi hálóját kellett leképeznem az adatbázisban. A rendszer központjában a Tournament entitás áll, amelyhez egy-a-többhöz kapcsolatban kötődik szinte minden további objektum. Ez a fődokumentum nemcsak alapadatokat tárol, hanem egy beágyazott konfigurációs objektumban tartja a lebonyolítás globális paramétereit is, mint például a pályák számát, a pihenőidőket és a mérkőzésszabályokat. A Category entitás felel egy adott kategória specifikus szabályaiért, így itt kerül meghatározásra a csoportból továbbjutók száma, a többszintű holtversenykezelési irányelvek, valamint a rájátszás mérete.

Tervezési szempontból az egyik legfontosabb döntésem a sporttechnikai és a pénzügyi-adminisztratív adatok szigorú szétválasztása volt. A Player entitás kizárólag a sportoló személyét és a helyszíni jelenléti állapotát reprezentálja. Tudatos tervezési döntés eredménye, hogy az alapverzióban egy játékos egyetlen kategóriához rendelődik hozzá közvetlenül a Player sémában, ami jelentősen leegyszerűsíti a lekérdezési logikát a kisebb versenyek esetében. Ezzel szemben a konkrét nevezési díjhoz, fizetési módhoz és számlázáshoz kapcsolódó adatok egy külön Entry gyűjteménybe kerültek. Ezt az adminisztrációs modult egészíti ki a PaymentGroup entitás, amely összefogja egy-egy klub vagy csapat játékosainak közös tranzakcióit, lehetővé téve a tömeges fizetéskezelést.

A rendszer központi logikai eleme vitathatatlanul a Match entitás. A túlbonyolított relációk elkerülése érdekében ezt a dokumentumot úgy terveztem meg, hogy egyaránt képes legyen kiszolgálni a csoportkörös és a kieséses mérkőzéseket. A modell egyetlen objektumon belül tárolja a tervezett és a tényleges kezdési, illetve befejezési időpontokat, az esetlegesen beosztott játékvezetőt, a mérkőzés aktuális státuszát, valamint a szettekre bontott eredményeket is.

A modell integritását a Group entitás támogatja, amely a résztvevők listázásán túl a visszalépések állapotkezelését és naplózását is végzi. A rendszer biztonságát és a

transzparenciát pedig az AuditLog entitás teszi teljessé, amely az adminisztratív lépések, valamint az állapotváltozások utólagos visszakövethetőségét garantálja.

Az adatmodell tervezése során tudatos hibrid stratégiát alkalmaztam a NoSQL-adatbázisok előnyeinek kihasználására. Míg a mérkőzések szetteredményeit beágyazott (embedded) dokumentumként kezelem a Match entitáson belül, mivel ezek életciklusa szorosan összefügg és mindig együtt kerülnek lekérdezésre, addig a Tournament és Player kapcsolatoknál a hivatkozással (reference) modellt alkalmaztam. Ez utóbbi biztosítja a horizontális skálázhatóságot és megakadályozza a dokumentumok méretének kontrollálatlan növekedését, ami a MongoDB 16 MB-os méretkorlátja miatt kritikus szempont.

4.3. API terv

A REST API végpontjainak tervezésekor abból az alapelvből indultam ki, hogy az interfész nem csupán alapvető adatszolgáltató réteg, hanem a rendszer elsődleges folyamatvezérlője is [9]. Emiatt az interfészt két markáns logikai részre bontottam. A hagyományos adatmódosító végpontokra és az erőforrás-specifikus üzleti műveleteket indító útvonalakra.

Egy kategória kezelése jól szemlélteti ezt az elvet. A kategória alapadatainak módosítása egy standard HTTP-híváson keresztül történik. Ugyanakkor a sorsolás lezárása és a csoportok automatikus legenerálása egy dedikált, operatív végponton keresztül hívható meg, hiszen ez nem egy egyszerű mezőfrissítés, hanem egy komplex, szerveroldali láncreakciót indító üzleti esemény. Hasonló elvet követtem a mérkőzések esetében is, ahol logikailag elkülönítettem a normál eredményrögzítést a speciális kimenetek, például a játék nélküli győzelem lezárásától, valamint a mérkőzések globális ütemezésétől.

A biztonság érdekében az interfész a nyilvános kijelzőnézet kivételével kizárólag hitelesített, védett végpontokból áll [7]. A hozzáférés-szabályozás nem a végpontok szintjén van statikusan rögzítve, hanem minden író és olvasó művelet előtt a dedikált szolgáltatási réteg ellenőrzi, hogy a kért erőforrás, például egy adott mérkőzés kategóriája, valóban a tokenből azonosított felhasználó tulajdonát képezi-e.

4.4. Kulcslogikák és algoritmusok

A rendszer valódi üzleti értékét és komplexitását nagyrészt azok a szerveroldali szolgáltatási modulok adják, amelyek a verseny tényleges lebonyolítását számítják és vezérlik. Tervezési alapelvként fektettem le, hogy ezek az algoritmusok, mint a párosításgenerálás, az eredményvalidáció, az állásszámítás, az ütemezés és az egyenes kieséses ág felépítése, különálló fájlokban éljenek, szigorúan elkülönítve az útvonalkezelőktől és az adatbázis-hozzáférési rétegtől. Ez a döntés egyrészt a kód karbantarthatóságát szolgálta, másrészt lehetővé tette a logikák izolált, egységtesztekkel történő ellenőrzését.

4.4.1. Csoportkörös mérkőzések generálása

A körmérkőzéses párosítások generálásához a klasszikus *polygon-rotációs* algoritmust implementáltam. Lényege, hogy a játékosokat egy tömbbe rendezem, az első elemet rögzítem, a többi pedig fordulónként elforgatom. Minden körben az i -edik és az $(n-1-i)$ -edik elem alkot párt (ahol n a tömb hossza). Ez garantálja, hogy a lehetséges párok pontosan egyszer játsszanak egymással.

Páratlan létszám esetén az algoritmus a lista végére egy üres értéket illeszt. Ha a rotáció során egy játékos ehhez az üres értékhez kerül, abban a körben pihenőt kap. Ezek a pihenőhetek egyenletesen oszlanak el a mezőnyben. Nagyobb létszámnál a rendszer részleges (csonka) körmérkőzést generál, megcélózva egy fix m mérkőzésszámot. Az aszimmetria elkerülésére ilyenkor *cirkuláns* gráfalapú párosítást alkalmazok. Az i -edik játékos minden $k=1\dots m/2$ eltolásra párosul az $(i+k) \bmod n$ indexű résztvevővel.

A rendszer a teljes és a csonka körmérkőzés közötti váltást dinamikus ajánlási logikával optimalizálja 6 résztvevőig teljes körmérkőzést, e felett pedig a mezőny méretétől függően 5 vagy 6 mérkőzésre korlátozott csonka rendszert javasol. Ez a skálázódás garantálja, hogy a terhelés nagyobb létszám esetén is az amatőr kereteken belül maradjon, miközben a *cirkuláns* párosítás fenntartja a sorsolás objektivitását.

4.4.2. Sorsoláslezárás és jelenlét alapján kialakított mezőny

A rendszerben a sorsoláslezárás jelenti azt a kritikus pontot, amikor a szoftver a benevezett játékosok listájáról átvált a ténylegesen induló mezőnyre. Ezt a helyszíni jelenlét-regisztráció előzi meg, mivel kizárólag a megerősített státuszú játékosok kerülhetnek be a mérkőzésgeneráló algoritmusba. A lezárás után a kategória egy új állapotba lép, ami szoftveres védelmet aktivál. Egy már megkezdett, azaz eredményekkel rendelkező csoportkörben a rendszer nem engedi újragenerálni a sorsolást, megelőzve az adatok inkonzisztenssé válását.

4.4.3. Eredményvalidáció

A tollaslabda eredménye egy szettekre bontott adatsor, aminek szigorú sportági szabályoknak kell megfelelnie. A serveroldalon futó validációs modul a Nemzetközi Tollaslabda Szövetség (BWF) szabályzata alapján [14] ellenőrzi a versenyenként konfigurálható paramétereket minden egyes beérkező szettnél. A rendszer megköveteli, hogy a győztes elérje a beállított célpontszámot, meglegyen a kötelező kétpontos különbség, pontegyenlőség esetén pedig érvényesüljön a maximális pontkorlát.

A validációs logika lekezeli az összes határesetet. Például egy 29-30-as vagy 28-30-as eredményt érvényesként fogad el, mivel a pontkorlát elérésekor az egyponos különbség, illetve a 30. pontnál elért kétpontos különbség is szabályos. Ugyanakkor a 27-30-as eredményt vagy a 21-21-es döntetlent visszautasítja, mivel az előbbi esetben a mérkőzés már 29-27-nél véget ért volna a kötelező kétpontos különbség teljesülésével. Ezt a folyamatot a mérkőzés egészét vizsgáló rutin egészíti ki, amely a megnyert szettek számát ellenőrzi, és megköveteli, hogy az összecsapás az egyik fél egyértelmű győzelmével végződjön. Mivel a validáció szigorúan a backend oldalon fut, a kliensoldali megkerülése vagy manipulálása lehetetlen.

4.4.4. Csoportállás és holtversenyek kezelése

Az állásszámítást egy háromszintű, lépcsős logikával oldottam meg, amely minden befejezett mérkőzés után automatikusan lefut. A rendszer elsődleges statisztikaként a győzelmi arányt, majd a győzelmek számát hasonlítja össze. Kétjátékos holtverseny esetén, ha az elsődleges mutatók egyeznek, a felek egymás elleni eredménye dönt. Többjátékos holtverseny, más néven körbeverés esetén a szoftver egy izolált szűkített tabellát épít fel. Ehhez dinamikusan leszűri az érintett játékosok egymás elleni mérkőzéseinek adatait, és ezen a szűkített halmazon számol új statisztikát. Ha ez sem hoz döntést, a teljes szettkülönbséget, majd a pontkülönbséget veszi alapul.

Amennyiben az algoritmus futása után is marad feloldatlan holtverseny, a rendszer nem kényszerít ki egy szoftveres, de sportszakmailag indokolatlan sorrendet. Ehelyett egy technikai jelzővel zárolja az érintett játékosokat, és figyelmezteti a szervezőt, hogy a végső sorrend kialakításához emberi, bírói döntésre van szükség.

4.4.5. Továbbjutók és egyenes kieséses szakasz generálása

A csoportkör lezárultával a szoftver a kategória konfigurációjának megfelelő számú továbbjutót emel ki. Ha a továbbjutási határon alakul ki feloldatlan holtverseny, az algoritmus megáll, és manuális döntést vár. Az egyenes kieséses ág felépítését egy dedikált generáló függvény végzi. A párosítások szigorú kiemelés alapján jönnek létre. Az i -edik helyezett az $(n-1-i)$ -edik helyezettel játszik az első körben. Ez a matematikai logika garantálja, hogy az első két helyen továbbjutó játékos csak a döntőben találkozhatson. Az algoritmus dinamikusan skálázódik a támogatott, kettő és harminckettő közötti ágméretekre. A mérkőzések előrehaladásával a győzteseket a rendszer automatikusan lépteti a következő körbe, és konfigurációtól függően az elődöntők veszteseiből bronzmérkőzést generál.

4.4.6. Ütemezési logika

A mérkőzések pályákhoz és időpontokhoz rendelését egy úgynevezett mohó algoritmussal [10] valósítottam meg. Az ütemező minden beosztandó mérkőzésnél végigiterál a szabad pályákon, és azt választja, amelyiken a legkorábbi kezdés garantálható. Egy mérkőzés becsült kezdési idejét az alapértelmezett kezdési időpont, a

pálya felszabadulásának ideje, valamint az érintett játékosok kötelező pihenőidejének lejártja közül a legkésőbbi időpont határozza meg.

A rendszer tehát külön időtengelyen követi a pályákat és a versenyzőket. Ha egy pálya felszabadul, de az egyik érintett játékos még a pihenőidejét tölti, az ütemező várakoztatja a mérkőzést. Ha két pálya pontosan ugyanakkor szabadul fel, az algoritmus azt részesíti előnyben, amelyiket addig kevesebbszer használták, ezzel egyenletesen elosztva a terhelést a csarnokban. Amennyiben a versenyhez játékvezetők is tartoznak, az ütemező őket is automatikusan rotálja, figyelembe véve az eddigi terhelésüket és a minimális bírói pihenőidőt. Mivel a valóságban a mérkőzések ritkán végződnek percre pontosan, beépítettem egy újraszámítási funkciót is. Ez a már lezárt találkozók tényleges befejezési idejéből kiindulva egyetlen utasítással frissíti a még várakozó mérkőzések becsült kezdési idejét anélkül, hogy a játékos- vagy pályabeosztást felborítaná.

Bár az ütemezési problémaosztály általános formájában NP-nehéz, a jelen rendszerben alkalmazott, korlátozott változatra, ahol a pihenőidők és a pályaszám determinisztikus korlátot jelent, nem adtam formális komplexitáselméleti bizonyítást, a választott mohó algoritmus az amatőr versenyek léptékén (jellemzően 4-12 pálya és 20-100 résztvevő) bizonyítottan szuboptimális, de a gyakorlatban kielégítő eredményt ad. Mivel a célfüggvény nem a matematikai értelemben vett abszolút minimum megtalálása, hanem a pályák üresjáratának minimalizálása a játékosok pihenőidejének kényszerfeltétele mellett, a mohó megközelítés $O(n \log n)$ komplexitása ideális egyensúlyt teremt a számítási sebesség és a használhatóság között.

4.4.7. Állapotkezelés és ütközésvédelem

A szoftver stabilitásának egyik kulcsa a szigorú állapotkezelés. Egy mérkőzést a rendszer például csak akkor enged elindítani, ha van hozzárendelt pálya és becsült időpont. Az indítási kérés beérkezésekor a backend két kritikus ütközésvizsgálatot futtat le. Ellenőrzi, hogy nincs-e már folyamatban lévő mérkőzés az adott pályán, illetve nem szerepel-e valamelyik játékos egy másik, párhuzamosan futó mérkőzésben. Mivel ezek az ellenőrzések szerveroldalon futnak, a felület esetleges hibája vagy szándékos manipuláció esetén is szerveroldalon ellenőrzött védelmet biztosítanak.

A lezárt versenyeknél alkalmazott állapotkezelés is ezt az elvet követi. A versenynap lezárultával az eredménymódosítás globálisan letiltásra kerül az adatbázisban. Ha mégis utólagos korrekcióra, például egy rosszul bemondott

szetteredmény javítására van szükség, a szervezőnek egy explicit zárolásfeloldó műveletet kell végrehajtania a korrekció idejére. Ez a megoldás ideális egyensúlyt teremt a rendszer rugalmassága és a már archivált adatok biztonsága között.

5. MEGVALÓSÍTÁS

A tervezési fejezetben lefektetett elvek gyakorlati alkalmazása során a cél egy jól karbantartható, moduláris kódbasis létrehozása volt. Ez a fejezet nem a korábban ismertetett algoritmusokat ismétli meg, hanem azt mutatja be, hogy a rendszer egyes rétegei hogyan épülnek fel a forráskódban, hogyan kommunikálnak egymással, és melyek azok a kritikus implementációs részletek, amelyek a rendszer stabilitását garantálják.

5.1. Backend modulok felelősségei

A szerveroldali alkalmazás belépési pontja végzi a környezeti változók betöltését, felépíti az adatbázis-kapcsolatot a Mongoose könyvtáron keresztül [5], regisztrálja az adatok feldolgozásáért felelős köztes rétegeket, majd sorban csatolja az összes útvonalcsoportot. A hibakezelő réteg az útvonalak után kerül regisztrálásra, így az összes kérés átmegy rajta, amennyiben valamelyik szolgáltatás kivételt dob. Az adatmodelleket tartalmazó könyvtárban található sémák nem csupán az adatszerkezetet írják le, hanem szigorú validációs szabályokat is érvényesítenek.

A mérkőzés entitás modellje a legösszetettebb. Egyetlen dokumentumban kezeli a résztvevőket, a versenykapcsolatot, az állapotot, az ütemezési adatokat, a szetteredményeket és a győztes meghatározását. Az eredménytípusokat egy enumerációs típus korlátozza, így adatbázisszinten is lehetetlen az érvénytelen állapot rögzítése [4].

A hitelesítési modul minden védett végpont elé beékelődik, kinyeri a hozzáférési tokent a kérés fejlécéből, ellenőrzi az aláírást és a lejáratot a JSON Web Token szabvány alapján [7], majd a dekódolt felhasználói azonosítót továbbítja a rendszernek. Érvénytelen vagy lejárt munkamenet esetén azonnal elutasítja a kérést, megakadályozva az illetéktelen hozzáférést. Az útvonalkezelőknél tudatos döntés volt az erőforrás-orientált struktúra alkalmazása és a műveletek kettéválasztása a REST elveknek megfelelően [9]. A hagyományos adatmódosító végpontok szigorúan elkülönülnek az operatív lebonyolítási lépésektől, mint amilyen a sorsolás lezárása vagy a csoportgenerálás. A rendszer üzleti logikáját a szolgáltatási réteg fogja össze, amely külön modulokban kezeli a körmérkőzéses párosításgenerálást, a többszintű holtversenyfeloldást, a sportágspecifikus eredményvalidációt [14], a mohó ütemezési algoritmust, valamint a tulajdonosi jogosultságok ellenőrzését.

5.2. Frontend fő nézetek és állapotkezelés

A kliensoldal belépési pontja inicializálja az alkalmazást, és egymásba ágyazva indítja el a hitelesítési, valamint az útvonalkezelő kontextust. A kliensoldali útvonalkezelés kialakításakor a saját implementációt egy specifikus mérnöki szükséglet indokolta. Lehetővé tette a MongoDB azonosítóinak (ObjectId) formátumához igazított előzetes validációs réteg közvetlen integrálását az útvonalkezelésbe. Ez a megoldás már a böngészőben kiszűri a hibás azonosítókat, megelőzve a felesleges és hibára futó szerveroldali hívásokat. Emellett a saját útvonalkezelő használata a böngésző beépített navigációs felületének (History API) mélyebb, alacsony szintű kontrollját is biztosította külső függőségek alkalmazása nélkül. Ez a megoldás jelentősen csökkentette a redundáns állapotfrissítések számát és a kliensoldali hibakezelés komplexitását.

A hitelesítési kontextus kezeli a munkamenet teljes életciklusát. Inicializáláskor megpróbálja visszaállítani a helyi böngészős tárhelyről a tokent, majd a háttérrendszeren keresztül ellenőrzi annak érvényességét. Lejárt munkamenet esetén automatikusan törli a tárolt adatokat, és átirányítja a felhasználót a bejelentkezési oldalra. A felület megalkotásakor a React komponensalapú felépítését követtem, így az ismétlődő vizuális elemeket, mint a dinamikus űrlapokat vagy az állapotjelzőket, újrahasznosítható egységekbe szerveztem, biztosítva a vizuális konzisztenciát és a reszponzív működést.

TABELLA

Csoportállás

A tabella csoportonként mutatja a helyezéseket, a holtverseny-döntő eredményét és a továbbjutási logikához fontos mutatókat.

Csoport A
4 játékos a tabellában

Rájátszás

HELY	JÁTEKOS	GYŐZELEM	LEJÁTSZOTT	GYŐZELEM %	SZETT DIFF	PONT DIFF	HOLTVERSENY
1	Róca Éva	3	3	1.000	6	31	rendben
2	Mészáros Anikó	2	3	0.667	0	8	rendben
3	Pintér Timea	1	3	0.333	0	-8	rendben
4	Vas Júlia	0	3	0.000	-6	-31	rendben

Tabella összesítő
Senior női egyéni

Csapatok	1
Értékelt játékosok	4
Döntetlen	0
Közös helyezések	0

Holtverseny szabály
A kategória konfigurációja alapján.

- Többfős holtverseny: csak mini-tabella
- Feloldhatatlan holtverseny: közös helyezés

1. ábra: Csoportállás nézet - holtverseny-feloldó mutatókkal

RÁJÁTSZÁS

Rájátszás és bronzmérkőzés

A playoff nézet a kategória teljes kiesései ágát mutatja. Innen generálható és továbbvihető az ágrajz, valamint gyorsan ellenőrizhető a döntő és a bronzmeccs állapota.

Csoport A
A csoportból induló playoff ág állapota.

ELŐDÖNTŐ

2026. 05. 07. 15:00

Varga Máté
Hegedűs Ákos
Győztes: Varga Máté
Lejátszott: 21-15, 21-14

2026. 05. 07. 15:50

Kis Roland
Szalai Bence
Győztes: Szalai Bence
Lejátszott: 18-21, 19-21

DÖNTŐ

2026. 05. 07. 17:30

Varga Máté
Szalai Bence
Győztes: Varga Máté
Lejátszott: 21-18, 21-19

BRONZMECCS

2026. 05. 07. 17:35

Hegedűs Ákos
Kis Roland
Győztes: Hegedűs Ákos
W.O. - -

Playoff összesítő
Férfi egyéni - junior főverseny

Összes playoff meccs	4
Várakozó	0
Fut	0
Befejezett	4

Kijelölt playoff meccs
Gyors eredményrögzítés a playoff nézetből.

Válassz egy meccset az ágrajzból.

2. ábra: Rájátszás és bronzmérkőzés nézet

KIJELZŐ

Kijelzős nézet

A futó és a következőként befejezett meccsek gyors áttekintése. Az oldal 30 másodpercenként automatikusan frissül.

Frissítve: 21:15:03 **Frissítés**

Most játszik (2)
Jelenleg futó meccsek pályánként:

1. PÁLYA
Kis Márk - **Tóth Dániel**
Női egyéni amatőr
Jkt: Szabó Gergely

2. PÁLYA
Teszt Áron - **Teszt Bence**
Hírbondolási kezdőkezdő
Jkt: Szabó Gergely

Következők (12)
Legközelebbi sorra kerülő, befejezett meccsek.

3. PÁLYA
Gál Zsófia - **Lakatos Dóra**
Női egyéni amatőr
20:25 Jkt: Kis András

5. PÁLYA
Biró Réka - **Gál Zsófia**
Női egyéni amatőr
20:35 Jkt: Szabó Gergely

3. PÁLYA
Teszt Áron - **Teszt Csaba**
Hírbondolási kezdőkezdő
21:18 Jkt: Németh Péter

3. PÁLYA
Papp Petra - **Biró Réka**
Női egyéni amatőr
22:05 Jkt: Fodor Tamás

3. PÁLYA
Tóth Márton - **Juhász Balint**
Hírbondolási kezdőkezdő
22:05 Jkt: Szabó Gergely

1. PÁLYA
Sipos Lilla - **Fekete Anna**
Női egyéni amatőr
21:35 Jkt: Németh Péter

1. PÁLYA
Sipos Lilla - **Lakatos Dóra**
Női egyéni amatőr
22:05 Jkt: Kis András

2. PÁLYA
Papp Petra - **Lakatos Dóra**
Női egyéni amatőr
21:35 Jkt: Fodor Tamás

2. PÁLYA
Teszt Csaba - **Teszt Dávid**
Hírbondolási kezdőkezdő
21:15 Jkt: Kis András

2. PÁLYA
Varga Levente - **Nagy Bence**
Férfi egyéni amatőr
21:35 Jkt: Kis András

3. PÁLYA
Fekete Anna - **Gál Zsófia**
Női egyéni amatőr
22:05 Jkt: Németh Péter

4. PÁLYA
Teszt Bence - **Teszt Dávid**
Hírbondolási kezdőkezdő
22:00 Jkt: Fodor Tamás

3. ábra: Nyilvános kijelzős nézet - automatikus frissítéssel

5.3. Integráció: adatáramlás, hibakezelés és validáció

A kliens és a szerver közötti zökkenőmentes adatáramlást egy dedikált kommunikációs réteg biztosítja, amely egységes interfészt nyújt a háttérrendszer felé. Ez a modul felelős a hálózati hívások felépítéséért, a hitelesítési token automatikus beillesztéséért a HTTP fejlécbe [7], valamint a válaszok előzetes feldolgozásáért.

A rendszer stabilitását és komplexitását legjobban a mérkőzés elindításának folyamata szemlélteti. Amikor a felhasználó a felületen kezdeményezi az indítást, a kliensoldal egy módosító kérést küld a szervernek. A háttérrendszerben elsőként a hitelesítési réteg azonosítja a felhasználót, majd a rendszer betölti a mérkőzést, a jogosultságkezelő modul pedig validálja a tulajdonosi viszonyt. A tényleges üzleti logika csak ezután veszi kezdetét. A szoftver ellenőrzi az érvényes indítási állapotot, a hozzárendelt pálya és időpont meglétét, majd lefuttatja a kritikus ütközésvizsgálatot. Ez garantálja, hogy a kijelölt pályán nincs folyamatban lévő találkozó, és az érintett játékosok sem szerepelnek párhuzamosan más mérkőzésben. Sikeres validáció esetén a mérkőzés futó állapotba kerül, az audit szolgáltatás naplózza az eseményt, a kliens pedig azonnal frissíti a felületet. Ütközés esetén a háttérrendszer megfelelő hibakóddal válaszol, amit a felület érthető magyar instrukcióvá alakít.

A lezárt versenyek adatbiztonságát az adatmodell egy dedikált logikai mezője garantálja, amely alapértelmezetten megakadályozza az eredmények utólagos módosítását. A szervezőnek lehetősége van egy explicit feloldási műveletre a kontrollált eredményjavítás érdekében, de ezt a folyamatot a rendszer minden esetben időbélyeggel és felhasználói azonosítóval naplózza, így a versenyintegritás utólag is visszakövethető marad. A felhasználói élményt támogatja a hibaüzenetek központosított fordítása is, amely a technikai kódok helyett azonnal értelmezhető magyar instrukciókat ad a szervezőnek, például egy szabálytalan szetteredmény rögzítési kísérletekor [14]. A rendszer robusztusságát a kétszintű validáció teszi teljessé. Míg a kliensoldal az alapvető formai ellenőrzéseket végzi, a biztonsági és üzleti szempontból kritikus vizsgálatok kizárólag a szerveroldalon futnak le, biztosítva az adatbázis konzisztenciáját még a felületi logika esetleges manipulálása esetén is.

5.4. A megvalósítás összeggése

A fejlesztés során a legnagyobb implementációs kihívást a kliens- és szerveroldali állapotok szinkronizálása jelentette, különösen a versenymérkőzések indításánál és az ütközésvédelemnél. A backend szolgáltatásalapú felépítése azonban sikeresen elszigetelte ezeket a komplex logikákat, és lehetővé tette, hogy a kritikus versenylogikák önálló, tesztelhető modulokban éljenek [3]. A frontend saját útvonalkezelővel és lokalizált hibakezeléssel biztosítja a reszponzív, azonnal reagáló felületet [1]. A két réteg közötti integráció tiszta szerkezetű. A kliensoldal közvetíti a felhasználó szándékát a szerver felé, amely a szigorú üzleti szabályok mentén hajtja végre a módosításokat.

6. TESZTELÉS ÉS VALIDÁCIÓ

Egy ilyen összetett, sok állapotot és összefüggő üzleti logikát kezelő rendszer esetében a tesztelés nem korlátozódhatott csupán a felület manuális kipróbálására [12]. A validáció során azt is bizonyítanom kellett, hogy az általam implementált algoritmusok szélsőséges bemenetek esetén is a tollaslabda szabályainak megfelelő, matematikailag helyes eredményt adnak [14].

6.1. Tesztstratégia

A megbízható működés igazolására egy háromszintű tesztelési stratégiát dolgoztam ki a szoftvermérnöki alapelvekkel összhangban [12]. Az első szintet a szerveroldali, logikai ellenőrzések jelentették. Ezek során az egyes szolgáltatási modulokat izoláltan vizsgáltam, hogy a megírt algoritmusok, például a holtversenyek feloldása vagy a kiemeléses ágrajz felépítése, helyes kimenetet adnak-e a beadott paraméterekre. A második szinten az integrációs és végponttól végpontig tartó teszteket végeztem el. Ehhez 17 dedikált automatizált tesztszkriptet írtam Node.js környezetben, amelyek a beépített fetch API-t és az assert/strict modult alkalmazzák külső tesztkeretrendszer bevonása nélkül. A szkriptek a böngésző megkerülésével, közvetlenül a REST API-t hívva szimulálnak komplett versenylebonyolítási folyamatokat. Lefedve a hitelesítést, a sorsolást, az eredményvalidációt, a holtversenyek feloldását, a rájátszást, az ütemezést és az auditnaplózást. Összesen 115 ellenőrzési ponttal vizsgálva az adatbázis konzisztenciáját és a végpontok hibátűrő képességét [12]. A harmadik szint a manuális, felhasználói szempontú tesztelés volt, amelyet a kliensoldali felületen hajtottam végre [1]. Ennek felgyorsítására megírtam egy adatkészlet-generáló szkriptet, amely másodpercek alatt felépít egy tesztfelhasználót, és feltölti az adatbázist különböző állapotú versenyekkel, így azonnal, reprodukálható környezetben tesztelhettem a vizuális elemeket és a navigációt.

6.2. Tesztesetek a követelményekhez kötve

A teszteseteket a követelményspecifikációban rögzített folyamatok mentén építettem fel, ellenőrizve a szoftver teljes életciklusát. A tesztelési folyamat úgynevezett füsttesztekkel indult. Ezek az automatizált folyamatok a rendszer legalapvetőbb, kritikus felületeinek gyors ellenőrzését végzik el annak megállapítására, hogy az alkalmazás egyáltalán funkcionálisan alkalmas-e a mélyebb, komplex validációra. Ezt

követően a tesztelés a hitelesítési modul és az alapvető jogosultságkezelés validálásával folytatódott [7]. Egy dedikált szkripttel bizonyítottam, hogy a rendszer szerveroldalon blokkol minden olyan kérést, amellyel egy hitelesített felhasználó egy másik szervező versenyadatait próbálná manipulálni. A lebonyolítási folyamat tesztelése a versenyek, kategóriák és játékosok rögzítésével, valamint a fizetési csoportok szinkronizációjával folytatódott.

Az adatok betáplálása után a rendszer legkritikusabb pontját, a sorsolás és a mérkőzésgenerálás pontosságát vizsgáltam. Az automatizált ellenőrzés igazolta, hogy a mérkőzésgeneráló algoritmus kizárólag a helyszínen ténylegesen megjelent, megerősített státuszú játékosokat veszi figyelembe a csoportok kialakításakor. A generált párosítások listáját programozottan vizsgáltam. A tesztek megerősítették, hogy nem jött létre duplikált párosítás, egyetlen játékos sem sorsolódott önmaga ellen, és a csonka körmérkőzéses beállításoknál a mérkőzésszám is kiegyensúlyozott maradt a mezőnyben.

Az eredményrögzítés validálása során szimuláltam mind a helyes, mind a sportági szabályoknak ellentmondó adatbevitelt [14]. A tesztek bizonyították, hogy a háttérrendszer azonnal visszautasítja az érvénytelen szetteket, megakadályozva ezzel a későbbi számítási anomáliákat. A rögzített eredmények alapján ellenőriztem a csoportállások frissülését, a továbbjutók automatikus kiválasztását, és az egyenes kieséses ág kiemelésalapú, hibátlan felépítését is. A szigorú validáció bizonyítására a 6.1. táblázat egy konkrét, formális tesztesetet mutat be az eredményrögzítő modul vizsgálatából.

6.1. táblázat: Formális teszteset - BWF szabályalapú eredményvalidáció

Paraméter	Érték / Leírás
Teszt célja	Szabálytalan mérkőzésekimenet elutasításának ellenőrzése
Előfeltételek	A mérkőzés állapota futó, a kategória beállítása: 21 pontig, maximum 30 pontig (pontkorlát).
Bemenet	Szetteredmény rögzítési kísérlet: 27-30
Elvárt kimenet	A szerver HTTP 400 (Bad Request) hibát ad, mivel 29-27-nél a mérkőzésnek már véget kellett volna érnie a kötelező kétpontos különbség elérésével. Az adatbázis nem frissül.
Tényleges kimenet	A rendszer elutasította a bevitelt, a felületen a megfelelő magyar nyelvű hibaüzenet ("Érvénytelen szetteredmény") jelent meg. SIKERES.

6.3. Kritikus helyzetek és "Edge case"-ek kezelése

Egy versenykezelő rendszer robusztusságát nem az ideális, hanem a szélsőséges esetekben mutatott viselkedése határozza meg [12]. A tesztelés során kiemelt figyelmet fordítottam a páratlan létszám kezelésére. A csoportkör-generálásnál ellenőriztem, hogy az algoritmus helyesen alkalmazza-e a virtuális pihenő pozíciót, és hogy ez a megoldás nem eredményez-e a felületen üres mérkőzéseket. Az összetett holtversenyek vizsgálatakor szándékosan olyan eredményeket generáltam, ahol több játékos között körbeverés alakult ki. A teszt igazolta, hogy a rendszer ilyenkor helyesen állítja fel a szűkített tabellát a szett- és pontkülönbségek alapján, illetve ha a holtverseny matematikailag feloldhatatlan, szabályosan leállítja az automatikus rangsorolást, és manuális döntést kér a szervezőtől [14].

A verseny közbeni ütközések szimulálása során a tesztek igazolták, hogy a szerveroldali ellenőrzések blokkolják azokat a kéréseket, amelyek ugyanazon a pályán vagy ugyanazon játékosal párhuzamos mérkőzést indítanak. Szintén letiltja az olyan találkozók indítását, amelyekhez még nincs időpont és pálya ütemezve. A játékvezetői

rotáció anomáliáit vizsgáló tesztek során megfigyeltem, mi történik, ha nincs elegendő játékező a sűrű menetrendhez. A rendszer ilyenkor nem omlott össze, hanem a minimális pihenőidők betartása mellett egyenletesen osztotta be a bírót, a fennmaradó mérkőzéseket pedig egyszerűen játékező nélkül ütemezte. A tesztelés kitért a versenyek életciklusának végére is. Egy befejezett állapotú verseny esetén a rendszer minden módosítási kísérletet blokkolt, az eredménymódosítás csak akkor volt lehetséges, ha az adminisztrátor explicit módon meghívta a zárolást feloldó folyamatot a korrekció idejére.

6.4. A tesztelés eredményeinek összessége

A lefolytatott automatizált szkriptek és a kiterjedt manuális tesztek alapján megállapítható, hogy a rendszer a specifikációban megfogalmazott minden követelményt megbízhatóan teljesít. A szoftver képes végigvinni egy tollaslabda-verseny teljes életciklusát, a játékosok regisztrációjától kezdve az ütemezésen át egészen a végeredmény hordozható formátumú exportálásáig. A legfontosabb tanulság és eredmény, hogy a szerveroldali interfész felépítése rendkívül stabilnak bizonyult. Egy adminisztratív webalkalmazásnál elkerülhetetlen, hogy a felhasználók hibás navigációval vagy rossz sorrendben végrehajtott műveletekkel teszteljék a határokat. A szerveroldali védelmek, a bemeneti adatvalidáció és a szigorú állapotkezelés garantálta, hogy a rendszer ilyen esetekben sem omlott össze, hanem kontrollált hibaüzenetekkel navigálta vissza a felhasználót a helyes munkafolyamathoz [12].

7. ÖSSZEGZÉS ÉS KITEKINTÉS

A szakdolgozatom elsődleges célja egy olyan webalapú tollaslabda-versenykezelő rendszer megtervezése és leprogramozása volt, amely a gyakorlatban is képes tehermentesíteni a kisebb, amatőr versenyek szervezőit. A fejlesztés során tudatosan kerültem egy túláltalánosított, minden sportágra alkalmazható menedzsment szoftver létrehozását. Ehelyett egy konkrét, jól körülhatárolt szakterületi problémára kerestem megoldást, fókuszálva a tollaslabda specifikus szabályrendszerére és a lebonyolítás során felmerülő tipikus, helyszíni kihívásokra.

7.1. Az elért eredmények összefoglalása

A fejlesztés eredményeként egy működőképes, háromrétegű webalkalmazást hoztam létre. A legfőbb eredménynek nem a felhasználói felület elkészítését tartom, hanem azt, hogy a rendszer a háttérben képes a versenyszervezés legbonyolultabb logikai feladatait automatizálni.

Sikerült megvalósítanom egy olyan munkafolyamatot, amely a versenykiírástól, a nevezések és a fizetési csoportok adminisztrálásától kezdve a helyszíni jelenlét-regisztráción át a tényleges mezőny kialakításáig vezeti a felhasználót. A szoftver önállóan, dedikált szolgáltatási modulokban végzi el a teljes és csonka körmérkőzéses párosítások legenerálását, a tollaslabda szabályai szerinti eredményvalidációt [14], a csoportállások és a bonyolult holtversenyek kiszámítását, valamint a továbbjutók alapján az egyenes kieséses ág felépítését. Mindezt egy olyan ütemező algoritmussal egészítettem ki, amely automatikusan osztja be a pályákat és a játékvezetőket a rendelkezésre álló időablakok és pihenőidők függvényében. Ezt a folyamatot a transzparencia érdekében auditnaplózással, az archiválhatóságért pedig hordozható adatexporttal támogattam meg.

7.2. A rendszer erősségei

A megoldásom legnagyobb erőssége a szigorú szakterületi fókusz. Mivel kifejezetten tollaslabdára készült, olyan részleteket is képes kezelni, mint a szettekre bontott ponthatárok és a többes körbeverések szűkített tabellás feloldása, amelyeket az általános táblázatkezelők vagy a dobozos sportmenedzsment szoftverek nem tudnak lefedni.

Architektúrális szempontból kiemelném a modularitást. A modern webes technológiák tudatos használata, valamint a backend szerkezetének logikai elkülönítése egy tiszta, jól karbantartható kódbázist eredményezett [12]. A rendszer robusztusságát a szerveroldali üzleti validációk adják, amelyek hatékonyan akadályozzák a pályautkőzéseket, az illetéktelen adatmódosításokat és a lezárt versenyek véletlen vagy jogosulatlan módosítását. Gyakorlati, fejlesztői szempontból nagy értéknek tartom a bemutató adatkészlet-generáló szkriptemet is, amely bármikor képes másodpercek alatt egy komplex, tesztelhető versenyállapotot felépíteni az adatbázisban.

7.3. Korlátok és hiányosságok (Tervezési kompromisszumok)

Mérnöki szemmel vizsgálva a rendszert fontos beszélni a korlátairól is. Ezek a hiányosságok nem a kód hibáiból erednek, hanem a szakdolgozati projekt szűkös időkerete miatt meghozott tudatos tervezési kompromisszumok.

A legfőbb korlát a jogosultságkezelés egyszerűsége. A rendszer jelenleg tulajdonosi logikán alapul, így a létrehozó látja és szerkesztheti a saját versenyét, de a szoftver nem tartalmaz finomhangolt, szerepköralapú hozzáférést. Ennek következtében nem lehet külön olvasási vagy dedikált játékvezetői fiókokat kiosztani egy eseményen belül. Emellett a szoftver kliens-szerver architektúrája folyamatos internetkapcsolatot igényel, így az elsődlegesen offline működés egy rossz lefedettségű sportcsarnokban jelenleg nem megoldott. A dokumentumkezelés terén az exportálás, bár az adatfeldolgozásra tökéletes, a hivatalos, kinyomtatható sorsolási és mérkőzéslapok generálása még várat magára. Végül a felhasználói felület, bár funkcionális és reszponzív, további ergonómiai finomításokat igényelne ahhoz, hogy piacképes terméké váljon.

7.4. Továbbfejlesztési lehetőségek

A fentebb említett korlátok adják a legkézenfekvőbb irányokat a szoftver jövőbeli továbbfejlesztéséhez. Az architektúra rugalmassága révén a rendszer viszonylag könnyen kiegészíthető lenne egy robusztusabb jogosultsági modellt kezelő modullal.

A versenylogika terén izgalmas kihívás lenne az algoritmusok kiterjesztése a vigaszágas rendszerekre, a páros és vegyes páros versenyszámok még specifikusabb kezelésére, illetve a kiemelési rendszer további finomítására. Az ütemező modul

jelenleg egy adminisztrátor által, gombnyomásra indítható dinamikus időbecslést használ. Ennek továbbfejlesztése egy valós időben, folyamatos háttérfolyamatként futó automatikus menetrend-kalkulátorra hatalmas hozzáadott értéket jelentene a szervezőknek. Hosszabb távon a rendszer kinyitható lenne a nagyközönség felé is a jelenlegi egyszerű nyilvános kijelzőnézet helyett publikusan megosztható linkekkel, élő online eredménykövetéssel, vagy akár külső nevezési űrlapok közvetlen integrációjával egy teljes körű szolgáltatássá nőhetné ki magát.

7.5. Személyes és szakmai tanulságok

A fejlesztési folyamat rámutatott, hogy egy sportágspecifikus szoftvernél az elméleti algoritmusok (mint például a tisztán matematikai sorsolás) önmagukban elégtelenek. A valós életbeli, operatív anomáliákat, például egy játékos késését vagy visszalépését zökkenőmentesen kell integrálni a számítási modellbe.

Sokáig dilemmáztam például a sorsolási rendszerek megvalósításán. Bár a svájci rendszer sportszakmai szempontból precíz, a projekt keretei között túl bonyolultnak ítéltam a stabil implementációját. Emiatt döntöttem a csonka körmérkőzés mellett, ami bár a gyakorlatban néha megosztó, informatikai szempontból az egyetlen járható út volt ahhoz, hogy hét fő feletti csoportoknál se emelkedjen kezelhetetlen szintre a mérkőzések száma. Felismertem, hogy egy amatőr versenyen nem az a prioritás, hogy hajszálpontosan tudjuk, ki a kilencedik vagy a tizedik helyezett, hanem az, hogy az első négy továbbjutót fair módon, a terhelés kiegyenlítése mellett válasszuk ki.

Technikai fronton a globális erőforrás-kezelés okozta a legtöbb álmatlan éjszakát. Egy korai fejlesztési fázisban az ütemező még kategóriánként elszigetelve kezelte a pályákat, ami a tesztelés során azonnal információs káoszhoz vezetett. Rá kellett jönnöm, hogy a pályákat és az időzítést globális szinten kell kezelnem, függetlenül attól, hogy éppen melyik kategória mérkőzése zajlik rajtuk. Ennek a logikának az újratervezése és a késési türelmi idő beépítése a jelenlét-regisztrációs folyamatba megtanított arra, hogy a szoftvernek rugalmasan kell kezelnie az emberi tényezőket is.

Külön kihívás volt a sportszerűség informatikai leképezése, például annak kezelése, ha valaki lesérül, vagy ha szubjektív okokból adja fel a küzdelmet. A pontarányok és a győzelmi statisztikák számolása során többször újraírtam a logikát, mire sikerült elérnem, hogy a holtverseny-feloldásnál csak a releváns mérkőzések mutatói döntsenek.

A projekt tanulsága nemcsak technikai jellegű volt. A hosszú fejlesztési ciklusok, az architekturális újratervezési fázisok és a kiterjedt tesztelési körök rávilágítottak arra, hogy egy komplex rendszer felépítése és stabil leszállítása komoly mérnöki fegyelmet, rendszerszemléletet és kitartást igényel.

7.6. Záró összegzés

Összességében kijelenthetem, hogy a kitűzött célt elértem. Létrehoztam egy webalapú, a gyakorlatban is bemutatható referencia-alkalmazást, amely bizonyítja, hogy a tollaslabda-versenyek legmonotonabb, emberi hibára leginkább hajlamos szervezési feladatai, mint a sorsolás, a számítások és az ütemezés, sikeresen algoritmizálhatók és automatizálhatók.

A projekt számomra nemcsak egy szoftver megírását jelentette, hanem egy komplex, valós üzleti probléma feltérképezését, a megfelelő informatikai architektúra megtervezését és a megoldás teljes körű, integrált megvalósítását. A rendszer a jelenlegi formájában stabil alapot képez, amely a közeljövőben például egy automatizált, valós idejű ütemező-kalkulátor integrálásával érdemi segítséget nyújthat a hazai amatőr tollaslabda versenyszervezőinek.

Irodalomjegyzék

- [1] Meta Open Source, React Documentation. Online. Elérhető: <https://react.dev/> Utolsó megtekintés: 2026. május 14.
- [2] OpenJS Foundation, Node.js Documentation. Online. Elérhető: <https://nodejs.org/api/> Utolsó megtekintés: 2026. május 14.
- [3] Express.js, Express Node.js web application framework. Online. Elérhető: <https://expressjs.com/> Utolsó megtekintés: 2026. május 14.
- [4] MongoDB Inc., MongoDB Documentation. Online. Elérhető: <https://www.mongodb.com/docs/> Utolsó megtekintés: 2026. május 14.
- [5] Mongoose, Mongoose Documentation. Online. Elérhető: <https://mongoosejs.com/docs/> Utolsó megtekintés: 2026. május 14.
- [6] Vite, Vite Documentation. Online. Elérhető: <https://vite.dev/guide/> Utolsó megtekintés: 2026. május 14.
- [7] M. Jones, J. Bradley, N. Sakimura, JSON Web Token (JWT), RFC 7519, Internet Engineering Task Force, 2015. Online. Elérhető: <https://datatracker.ietf.org/doc/html/rfc7519> Utolsó megtekintés: 2026. május 14.
- [8] T. Bray, Ed., The JavaScript Object Notation (JSON) Data Interchange Format, RFC 8259, Internet Engineering Task Force, 2017. Online. Elérhető: <https://datatracker.ietf.org/doc/html/rfc8259> Utolsó megtekintés: 2026. május 14.
- [9] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, PhD. Dissertation, University of California, Irvine, 2000. Online. Elérhető: <https://roy.gbiv.com/pubs/dissertation/top.htm> Utolsó megtekintés: 2026. május 14.
- [10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, és C. Stein, Új algoritmusok (Introduction to Algorithms), 3. kiadás. Budapest: Sclor Kiadó, 2011.
- [11] D. E. Knuth, The Art of Computer Programming, Volume 2: Seminumerical Algorithms, 3rd ed. Reading, Massachusetts: Addison-Wesley, 1997.
- [12] I. Sommerville, Software Engineering, 10th ed. Pearson, 2015.
- [13] MDN Web Docs, MDN Web Docs (Mozilla Developer Network) Web API and JavaScript Reference. Online. Elérhető: <https://developer.mozilla.org/> Utolsó megtekintés: 2026. május 14.
- [14] Badminton World Federation, Laws of Badminton / BWF Statutes. Online. Elérhető: <https://corporate.bwfbadminton.com/statutes/> Utolsó megtekintés: 2026. május 14.
- [15] Szegedi Tudományegyetem Informatikai Intézet, Szakdolgozat és diplomamunka formai követelmények. Online. Elérhető: <https://www.inf.u-szeged.hu/hallgatoknak/diplomamunka-szakdolgozat/szerkezeti-es-formai-kovetelmenyek> Utolsó megtekintés: 2026. május 14.
- [16] OWASP Foundation, OWASP Cheat Sheet Series JSON Web Tokens. Online. Elérhető: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html Utolsó megtekintés: 2026. május 14.

Nyilatkozat

Alulírott Gál Gergő Károly, gazdaságinformatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem Informatikai Intézet Képfeldolgozás és Számítógépes Grafika Tanszékén készítettem, gazdaságinformatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Dátum: 2026. május 14.

Aláírás: _____

Mesterséges intelligencia használatáról szóló nyilatkozat

Alulírott Gál Gergő Károly kijelentem, hogy a szakdolgozat és a hozzá tartozó szoftver elkészítése során generatív mesterséges intelligencia alapú eszközöket kizárólag támogató jelleggel vettem igénybe.

Az MI-eszközöket elsősorban nyelvi és stilisztikai lektorálásra, fejlesztés közbeni hibaüzenetek értelmezésére, már elkészült kódrészletek olvashatóságának javítására, valamint formázási és szerkesztési feladatok támogatására használtam.

Kijelentem, hogy a rendszer architekturális megtervezését, az adatmodell kialakítását, a sorsolási, validációs, állásszámítási és ütemezési logikák szakmai döntéseit, valamint a dolgozat érdemi tartalmát önállóan dolgoztam ki. Az MI által javasolt tartalmakat minden esetben ellenőriztem, szükség szerint módosítottam, és a szakdolgozat végső szöveges, illetve szoftveres tartalmáért teljes felelősséget vállalok.

Dátum: 2026. május 14.

Aláírás: _____

Köszönetnyilvánítás

Ezúton szeretnék köszönetet mondani konzulensemnek, Dr. Bodnár Péter egyetemi adjunktusnak a folyamatos szakmai iránymutatásért. Köszönöm a konzultációkra szánt idejét, a hasznos tanácsait és a türelmét, amellyel végigkísérte és támogatta a szoftverfejlesztés, valamint a dolgozat elkészítésének folyamatát. Az ő iránymutatása és konstruktív kritikái nélkül ez a munka nem nyerhette volna el a végleges formáját.

Külön hálával tartozom Szűcs Zoltánnak, a Talentum Tollaslabda Egyesület vezetőedzőjének. Ő volt az, aki megadta a kezdőlökést ehhez a projekthez, és akinek a gyakorlati tapasztalataira végig támaszkodhattam. Köszönöm, hogy időt szakított a rendszer véleményezésére, és rámutatott azokra a valós, életszerű versenyszervezési problémákra, amelyek fejlesztői oldalról kevésbé lennének nyilvánvalók.

Végül, de nem utolsósorban szeretném megköszönni a családomnak és a barátaimnak a türelmet és a kitartó biztatást. Támogatásuk rengeteget jelentett a munka előrehaladása és a rendszer véglegesítése során.